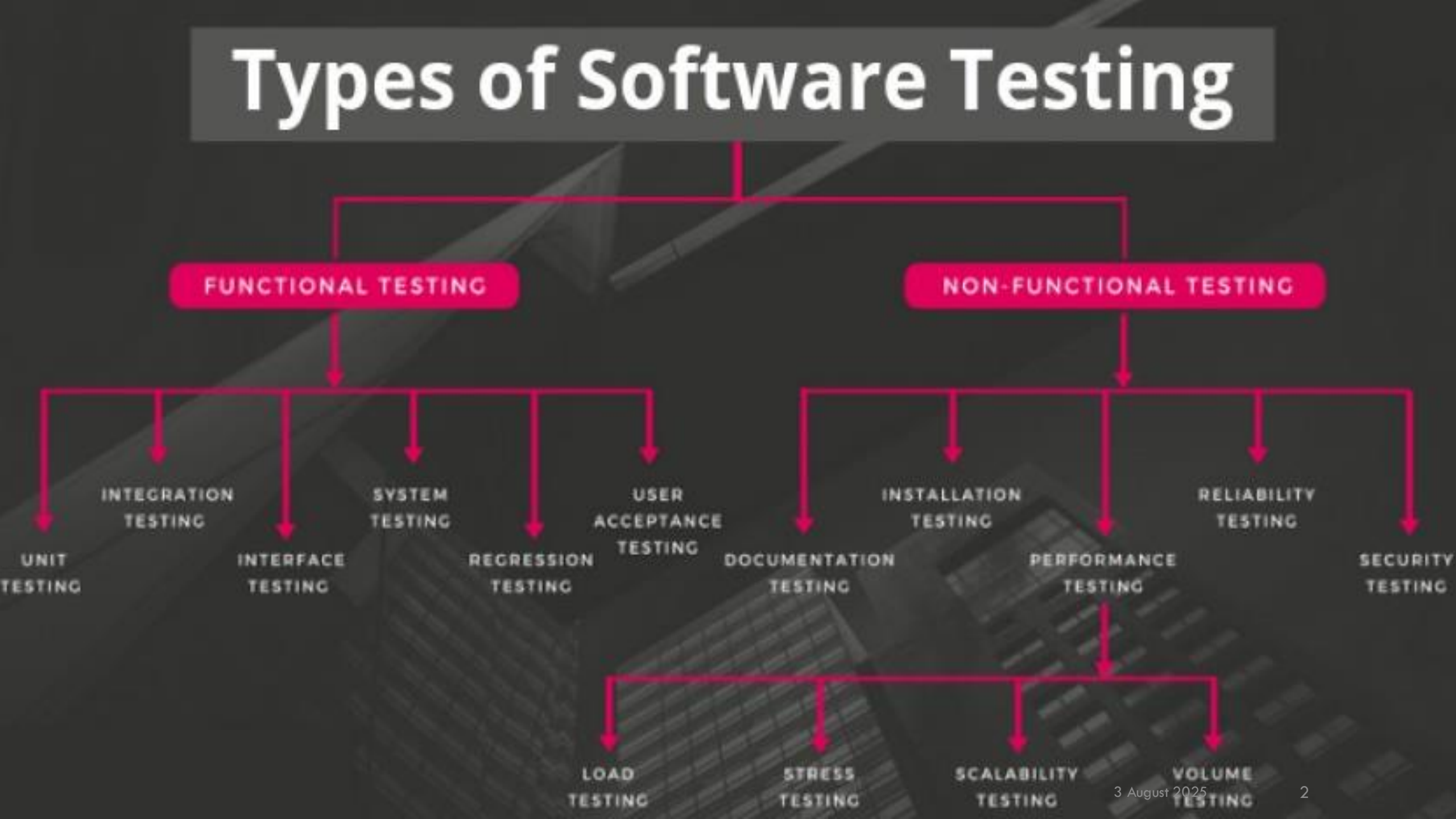


ASD06 Software Testing

Johan Bontes

CSC3003S, 2023

With credit to Sarina Till for original slides


Types of Software Testing
FUNCTIONAL TESTING
NON-FUNCTIONAL TESTING
UNIT TESTING
INTEGRATION TESTING
INTERFACE TESTING
SYSTEM TESTING
REGRESSION TESTING
USER ACCEPTANCE TESTING
DOCUMENTATION TESTING
INSTALLATION TESTING
PERFORMANCE TESTING
RELIABILITY TESTING
SECURITY TESTING
LOAD TESTING
STRESS TESTING
SCALABILITY TESTING
VOLUME TESTING

No silver bullet

Table 20-2 Defect-Detection Rates

Removal Step	Lowest Rate	Modal Rate	Highest Rate
Informal design reviews	25%	35%	40%
Formal design inspections	45%	55%	65%
Informal code reviews	20%	25%	35%
Formal code inspections	45%	60%	70%
Modeling or prototyping	35%	65%	80%
Personal desk-checking of code	20%	40%	60%
Unit test	15%	30%	50%
New function (component) test	20%	30%	35%
Integration test	25%	35%	40%
Regression test	15%	25%	30%
System test	25%	40%	55%
Low-volume beta test (<10 sites)	25%	35%	40%
High-volume beta test (>1,000 sites)	60%	75%	85%

Source: Adapted from *Programming Productivity* (Jones 1986a), "Software Defect-Removal Efficiency" (Jones 1996), and "What We Have Learned About Fighting Defects" (Shull et al. 2002).



ASD06
Software Testing

A. Defensive programming

B. Automated testing

C. Unit Tests

D. Integration

E. Regression

F. User Acceptance

G. Security

Defensive programming

Const correctness

```
class user {
  void setName(const string& newName);
  string getName() const;
```

input validation

```
void setName(const unsigned uid; const string& newName) {
  if (uid >= size(Users)) { ....
  if (sameStr(newName, getName(id)) { ....
  if (newName == "") { ...
  if (size(NewName) < 2) { ....
```

assertions

```
void setName(const unsigned uid; const string& newName) {
  assert(Users); //user db must be initialized
  assert(isValid(Users)); //sanity check of User db passes
  assert(newName != nullptr); //no! must run at runtime
```

error handling

```
void setName(const unsigned uid; const string& newName) {
  try {
      internalSetName(uid, newName);
  } catch ...
```

resource man.

```
String readFile(String name) {
    File file = new File(name);
    try { do stuff with file }
    finally { file.close(); }
```

```
void rF(const string& name)
  auto file = ifstream(name)
  do stuff with file
} //RAII: ifstream destroyed
```

Const correctness

- Java: JSR 305+308 [1]

  `@Readonly @Immutable @Mutable`

- C++/Rust, etc:
  - const **methods** cannot change their objects
  - const **objects** cannot be changed
  - const **parameters** are fixed in the body

```
public String getName(@ReadOnly Person this) {
    return name;
}

@ReadOnly Person person = new Person();
person.getName(); //succeeds
person.setName("test"); //does not compile
```

```
string Person::getName() const {
  return name;
}

const auto person = new Person();
person.getName(); //succeeds
person.setName("test"); //does not compile
```

```
void doStuff(const int uid) {
  uid++; //does not compile
  auto nextId = uid + 1; //fine
}
```

Input validation

Must run in release code ≠ assertions

input within bounds
```
if (id < size(P)) { ...
```

object valid
```
if (P && P.isOpen()) { ...
```

container big enough
```
if (size(A) >= size(input)) {
```

no SQL injection
```
sqlSafe = sanitize(sql);
```

use parameterised queries instead!

no HTML injection
```
displayTxt = sanitize(userTxt);
```

Never display user input Always sanitize display text

Escaping is hard, never roll your own, use standard solutions

Assertions

Condition should never occur in production code

Can be as slow as you want

Only works in DEBUG code

Don't put action code in assertions `assert(DoAction());`
`//removed in release`

Use assertions for pre- & postconditions

Design by contract (Meyer 1997, C++26)

```
func Velocity(lat, lon, height: float): float;
  assert(-90 <= lat and lat <= 90); //pre
  assert(0 <= lon and lon < 360);
  assert(-500 <= height and height <= 75000);
  ...
  assert(0 < result and result <= 600); //post
  return result;
```

Meyer, Object-Oriented Software Construction, 1997
https://en.cppreference.com/w/cpp/language/contracts.html

Assertions

Assertions break debug state

if so, put assertion in check

```
func Velocity(lat, lon, height: float): float;
  #ifdef _DEBUG
  if  (not(-90 <= lat and lat <= 90)) {
    assert(-90 <= lat and lat <= 90);
```

Exactly the same code in `if (not()) {` statement as in assertion

Breakpoint: program state is preserved

Meyer, Object-Oriented Software Construction, 1997
https://en.cppreference.com/w/cpp/language/contracts.html

Error handling

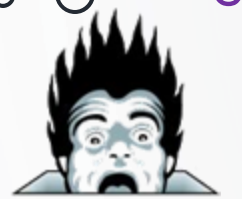
2½ schools of thought

- Exceptions
  - Zero overhead in happy path
  - Lots of overhead in error path
- Error codes
  - little overhead in happy path
  - little overhead in error path
  - much more difficult to get right!
- Handle errors on the spot
  - Needs a crystal ball

Exceptions

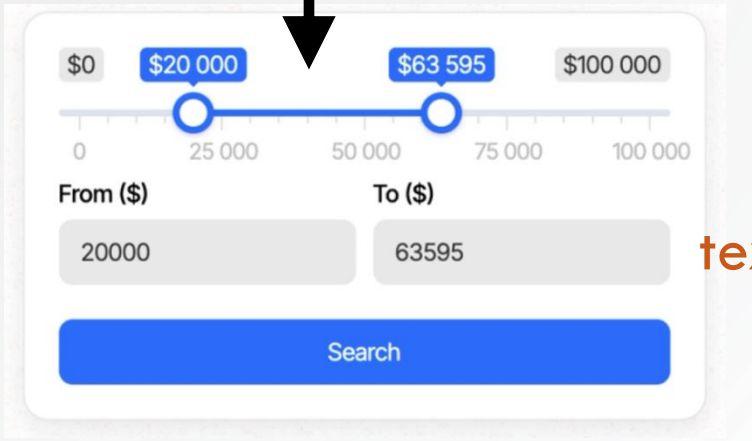
try catch also: (try except)

```
try { a = file.getData(); ... }
catch (SomeException e) {
  //ignore it
  //do something useful
  editBox.textColor = clRed;
} finally {
  file.close();
}
```



never silently drop errors

sliders cannot be in error



text boxes can

Do not leak exceptions to the user:

Cooper: about face: [Error messages are idiotic and should never be used] [1]

Instead prevent the user from making errors in the first place

Bonus check: compiler warnings are your friend

C++: compile with full warnings enabled

```
MSVC: /W4
Clang: -Wall –Wextra –Wpendantic –Weverything
GCC: -Wall –Wextra
```

Python:

```
import warnings
warnings.simplefilter('always')
```

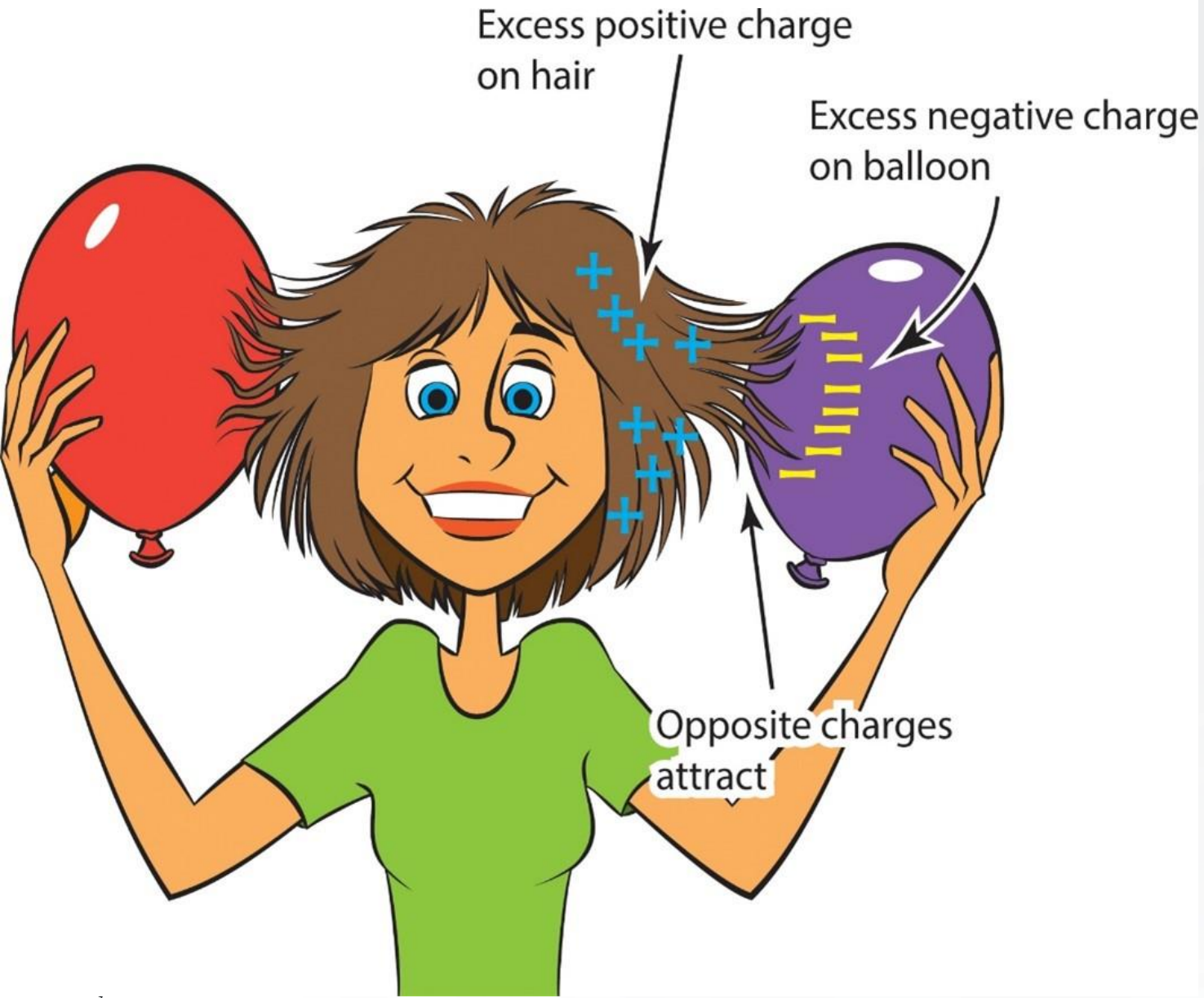
Java: differs per IDE, check documentation

NetBeans

// Project properties → Build → Compiling → Show warnings = true

// Tools → Options → Java → Compiler → Warnings = All

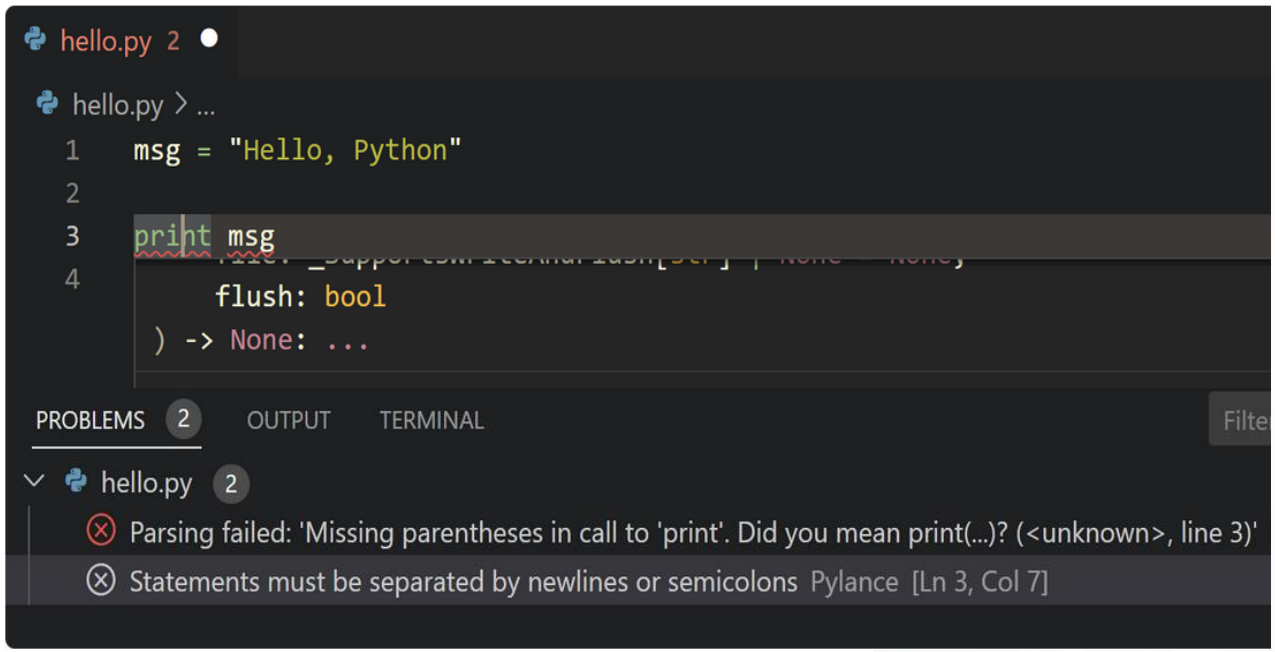
Static analysis



ASD06
Software Testing

A. Defensive programming
B. Automated testing
C. Unit Tests
D. Integration
E. Regression
F. User Acceptance
G. Security

Automated testing: static analysis



- Lint: 1978 [2] *esp. useful for Python*
- Checks for 'smells' + bugs in code
  - smells: bad programming practice [1]
    - aka: known antipatterns
    - e.g.: duplicated or complex code, poor naming, global vars
  - bugs: code matching known bug patterns
    - infinite loops, division by zero, ...
    - formal proofs are NP-Hard => rarely feasible

[1] Fowler: *Refactoring: Improving the Design of Existing Code*, 1999
[2] S. Johnson *Lint, a C Program Checker*. Comp. Sci. Tech. Rep, 1978: https://web.archive.org/web/20220123141016/https://citeseerx.ist.psu.edu/viewdoc/download?doi=10.1.1.56.1841&rep=rep1&type=pdf

Automated testing: code sanitizers

- Google 2012: AddressSanitizer
- Works by instrumenting source code or injecting into compiled code
- Many checkers
  - AddressSanitizer: dangling pointers, read past end, use after free
  - MemorySanitizer: uninitialized memory
  - UBSan: Undefined behavior
  - LeakSan: memory allocated, but not freed
  - ThreadSan: deadlocks, data races

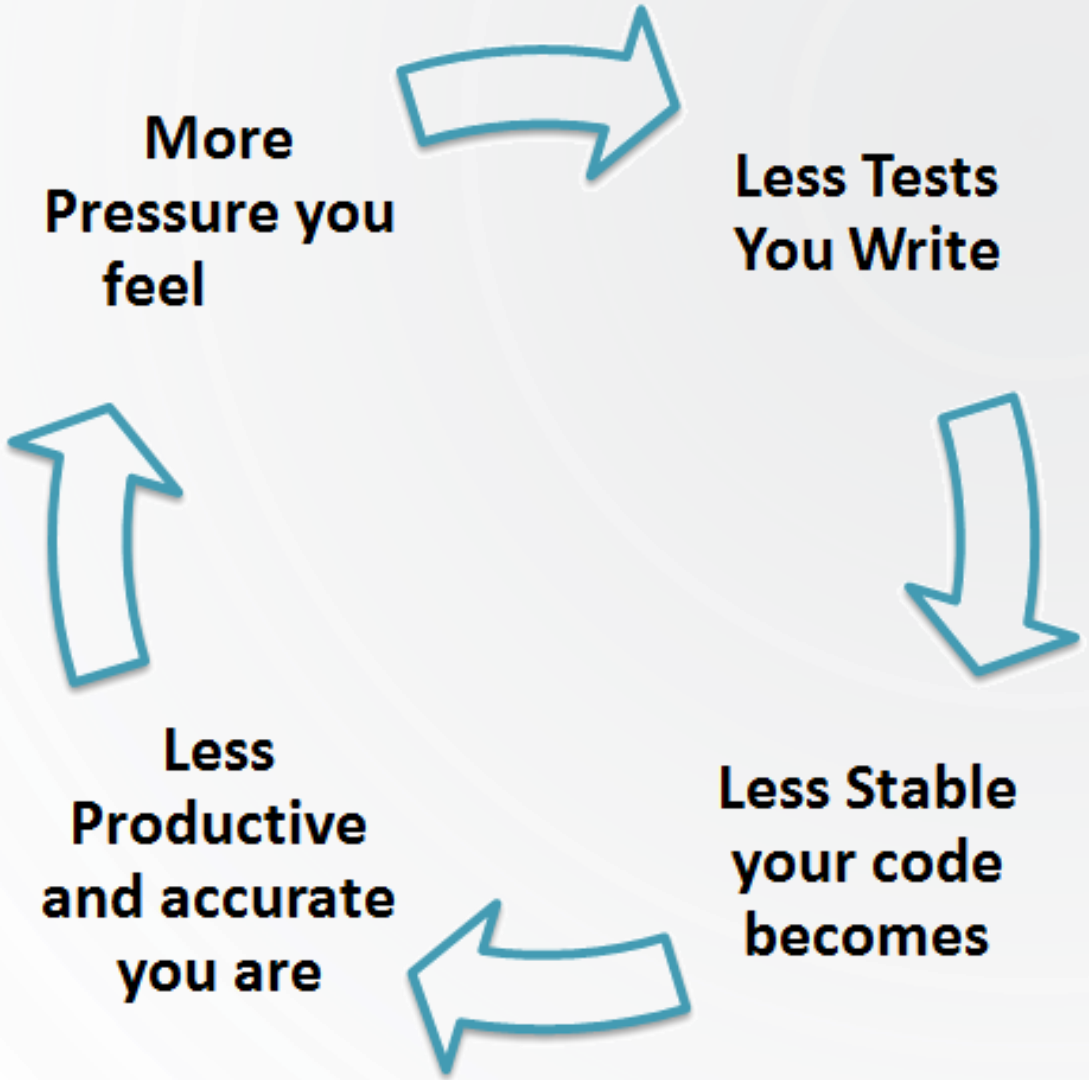
Automated testing: code sanitizers

Demo

Automated testing: fuzzing

- Feeds randomised input into program [1]
  - Checks for error handling
  - See if program misbehaves
  - Security checks
- Black-box vs White-box
- Tools for most languages, e.g.:
  - Python: `pip3 install atheris`
  - Java: https://github.com/CodeIntelligenceTesting/jazzer
  - C++: https://llvm.org/docs/LibFuzzer.html

[1] B.. Miller, et al, *An Empirical Study of the Reliability of UNIX Utilities*, Communications of the ACM, 1990



CSC3003S ASD07 Software Testing

A. Defensive programming
B. Automated testing
C. Unit Tests
D. Integration
E. Regression
F. User Acceptance
G. Security

Unit tests

- Tests source code in isolation
  - unit = small section of code, often a class or function, that can be tested in isolation
- Frameworks for most languages
  - Java: Junit
  - Python: unittest, ...
  - C++: Catch2, ...
- Separate module with test cases
- Goal is 100% test coverage
- Test coverage:
  - All code *(automatic)*
  - All possible inputs *(hard)*
  - Python: coverage https://coverage.readthedocs.io/en/7.9.2/

```
#include <catch2/catch_test_macros.hpp>

unsigned factorial(const unsigned n) {
  assert(n < 13); //why this assert?
  if (n <= 1) { return n }
  else return factorial(n) * n; //code smell
}

//test cases usually in separate file
//one file per file to be tested
TEST_CASE( "Factorials are computed", "[factorial]" ) {
  REQUIRE( factorial( 1) == 1 );
  REQUIRE( factorial( 2) == 2 );
  REQUIRE( factorial( 3) == 6 );
  REQUIRE( factorial(10) == 3'628'800 );
  REQUIRE( factorial(13) == 6’227’020’800 ); //fail
```

Unit tests

- Tests source code in isolation
  - unit = small section of code, often a class or function, that can be tested in isolation
- Frameworks for most languages
  - Java: Junit
  - Python: unittest, ...
  - C++: Catch2, ...
- Separate module with test cases
- Goal is 100% test coverage
- Test coverage:
  - All code *(automatic)*
  - All possible inputs *(hard)*
  - Python: coverage https://coverage.readthedocs.io/en/7.9.2/

```
-------------------------------------------
Testing Binary Search
-------------------------------------------
test.cpp:9
...........................................
test.cpp:18: FAILED:
REQUIRE( binary_search(arr, 176) == 0 )
with expansion:
-1 == 0
===========================================
test cases: 1 | 1 failed
assertions: 5 | 4 passed | 1 failed
```

TDD: test driven devel.

1. Write code with no implementation
2. Write test for code => will fail
3. Implement code so (more) tests passes
4. Before adding code: add tests => fail
5. Add more code so tests passes

Q: do we have full coverage?

6. Make tests comprehensive

```
1: unsigned factorial(const unsigned n){return n}

2: TEST_CASE("Do factorials", "[factorial]" ) {
  REQUIRE( factorial( 1) == 1 );
  REQUIRE( factorial( 2) == 2 );
  REQUIRE( factorial( 3) == 6 );
  REQUIRE( factorial(10) == 3'628'800 );

3: unsigned factorial(const unsigned n) {
  auto f = n;
  for(auto i = 1; i <= n; i++) f *= i;
  return f;
}

4: REQUIRE( factorial(13) == 6'227'020'800 );
   REQUIRE( factorial(21) == {} );

5:optional<uint64_t> factorial(const uint64_t n) {
  if (n > 20) { return {}; //empty result }
  assert(n <= 20); //defensive programming

6: for (unsigned i = 21; i < UINT32_MAX; i *= 2) {
     REQUIRE( factorial( i) == {} );
```

What if class depends on others

What if your code depends on a component not yet implemented?

- mock objects, dummy databases, dummy APIs
  - Java: mockito: https://site.mockito.org/
  - Python: `unittest.mock`: https://realpython.com/python-mock-library/

```
from unittest.mock import Mock
json = Mock()
json.loads('{"key": "value"}') #will magically work

json.loads.assert_called()
json.loads.assert_called_once()
json.loads.assert_called_with('{"key": "value"}')
json.loads.assert_called_once_with('{"key": "value"}')
```

More mocking

```
from datetime import datetime
from unittest.mock import Mock

datetime = Mock()

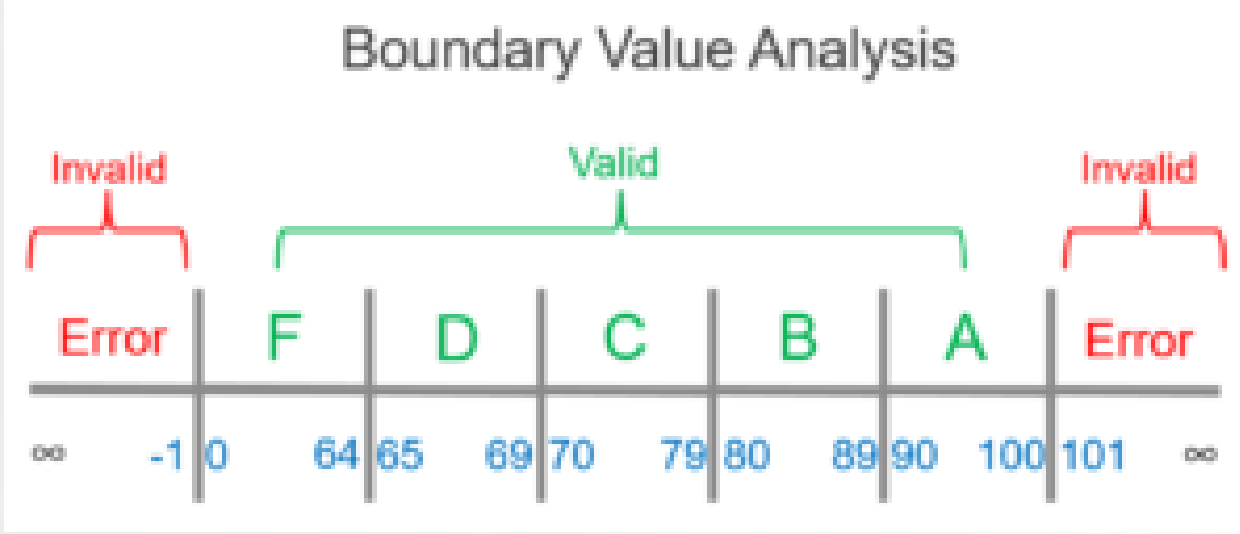
def is_weekday(d):
    today = d.today()
    # Python's datetime library treats Monday as 0 and Sunday
as 6
    return (0 <= today.weekday() < 5)

# Mock .today() to return Wednesday
datetime.today.return_value = wednesday
# Test Wednesday is a weekday
assert is_weekday(datetime)

datetime.today.return_value = sunday
# Test Sunday is not a weekday
assert not is_weekday(datetime)
```

Selecting Test Cases

- Partition Testing:
  group possible inputs and outputs into categories (partitions)
  → look for edge cases
- Guideline Testing – choose test cases based on known failure points





Unit testing resources

- https://medium.com/@clarkjasonngo/junit-test-with-maven-in-vscode-7546eabd50f7
- https://www.guru99.com/unit-testing-guide.html



ASD06 Software Testing

A. Defensive programming

B. Automated testing

C. Unit Tests

D. Integration

E. Regression

F. User Acceptance

G. Security



Unit test vs Integration Test

Integration Testing

Test interactions between modules and classes

Same as unit tests, but on a higher level

Unit testing: class/function in isolation.

	executed	failed
iterations	1	0
requests	4	0
test-scripts	4	0
prerequest-scripts	0	0
assertions	4	0

total run duration: 650ms

total data received: 1.91KB (approx)

average response time: 80ms [min: 10ms, max: 265ms, s.d.: 106ms]

C++: https://softwarepatternslexicon.com/patterns-cpp/16/6/
Java: https://www.programmingempire.com/integration-testing-with-junit/
Python: https://coderivers.org/blog/python-integration-testing/
https://www.guru99.com/integration-testing.html

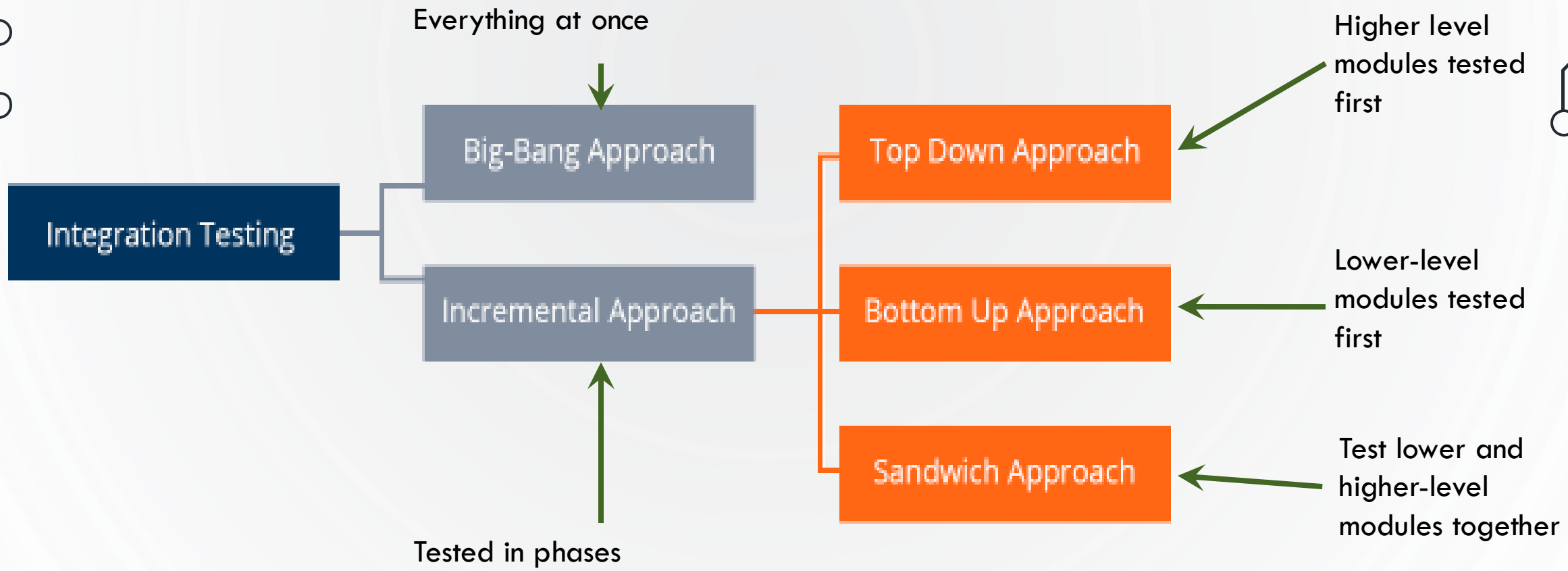
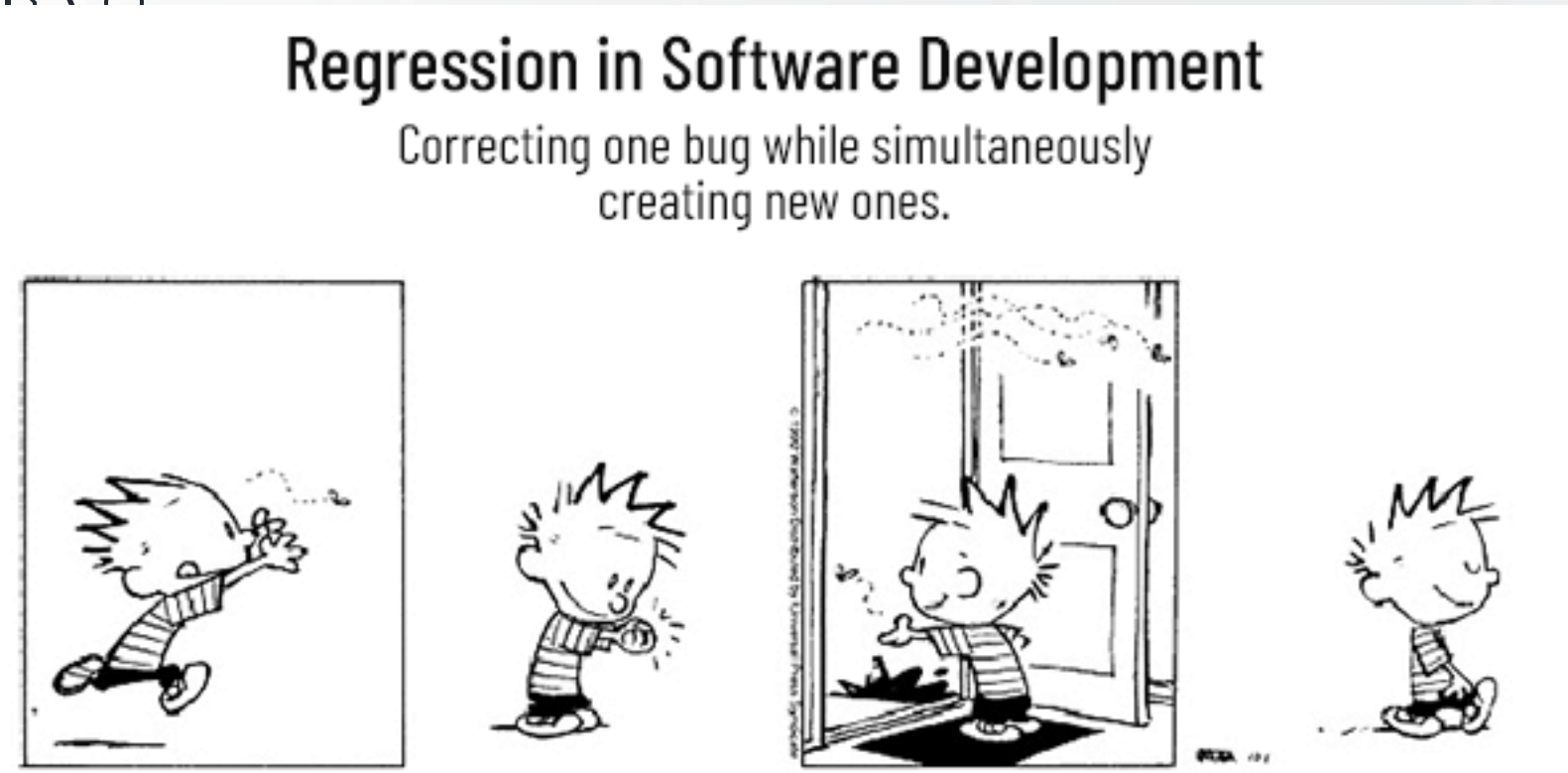
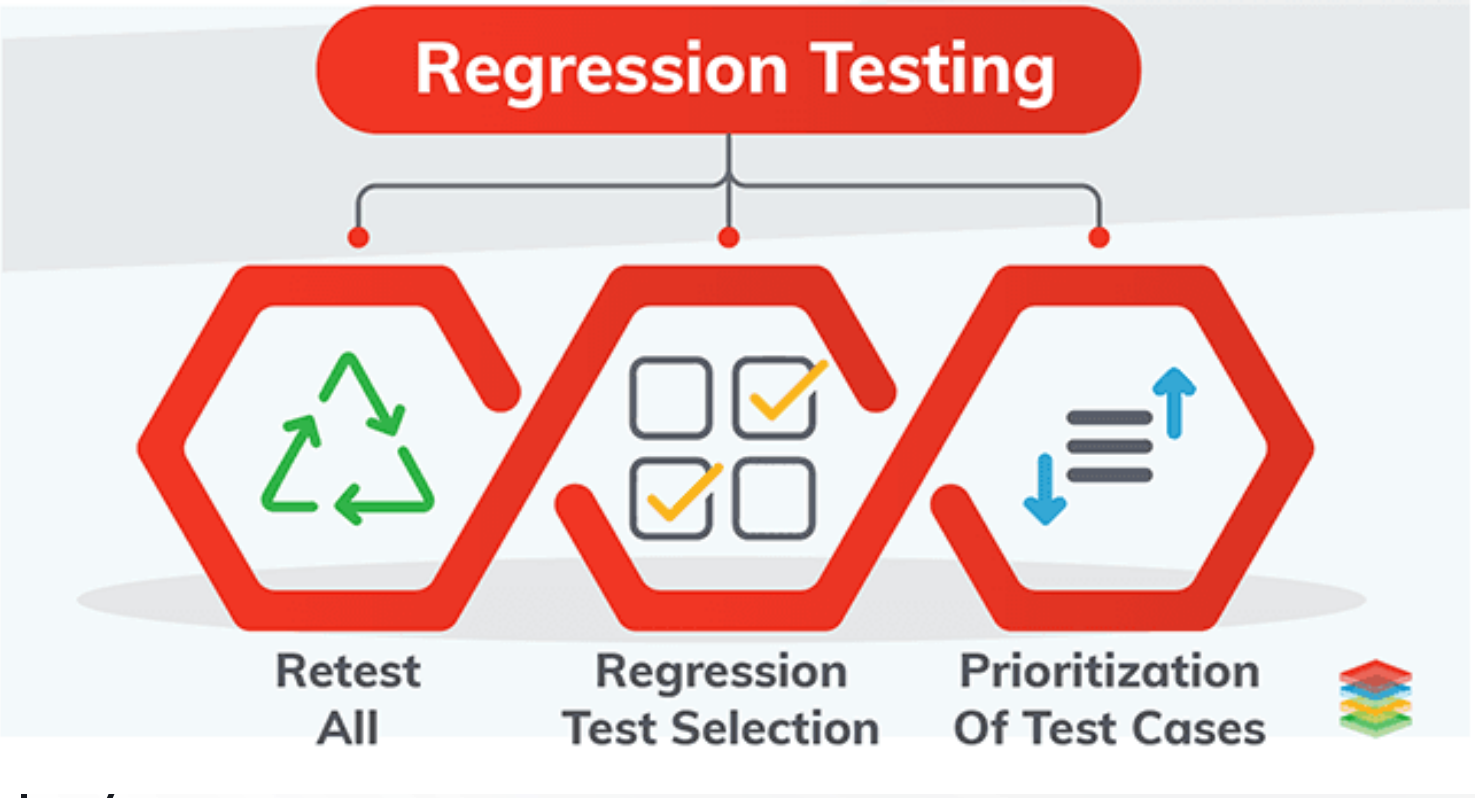


Image: https://qatestlab.com/resources/knowledge-center/big-bang-testing/



ASD06
Software Testing

A. Defensive programming
B. Automated testing
C. Unit Tests
D. Integration
E. Regression
F. User Acceptance
G. Security



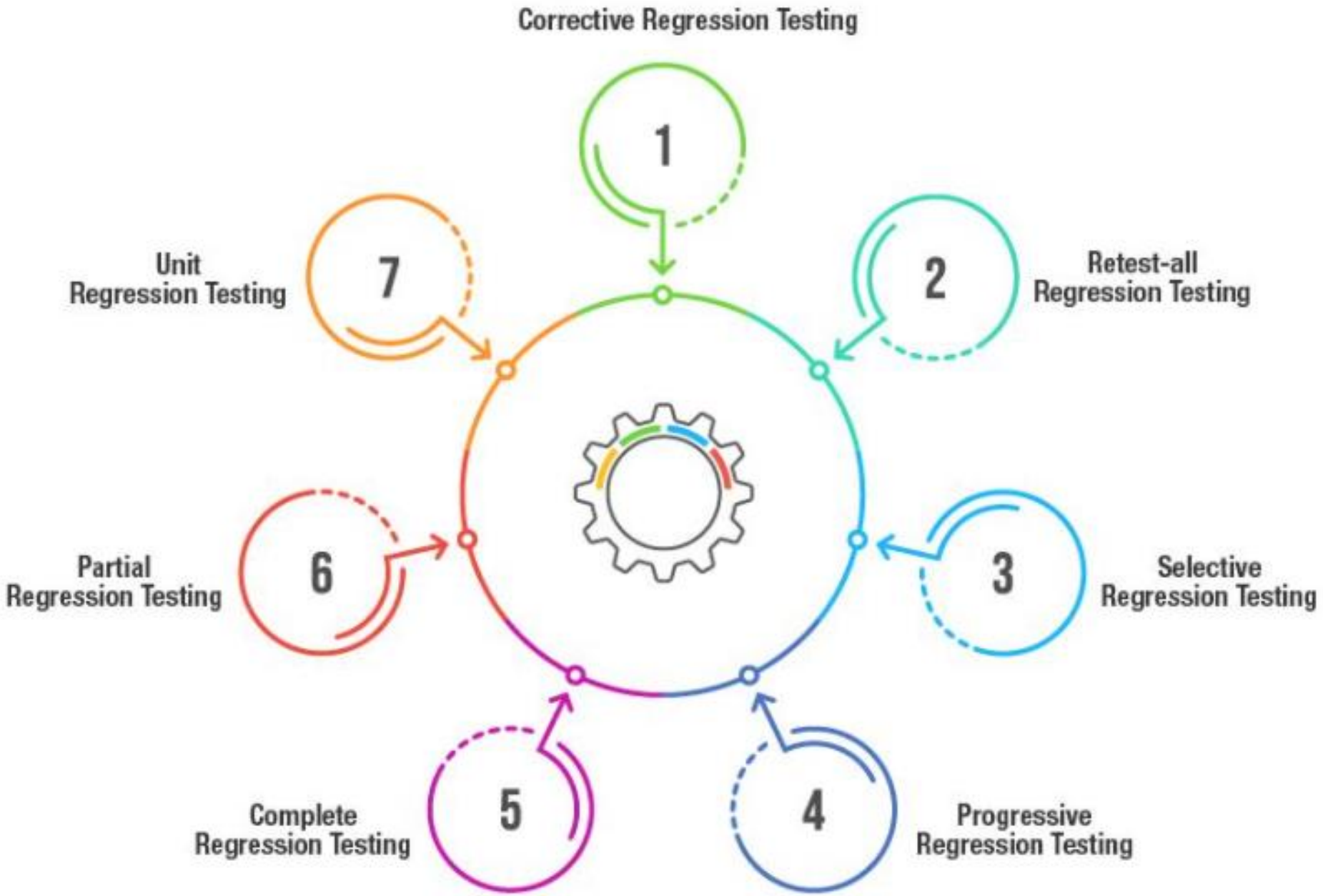
Add new code?

Rerun all your old tests

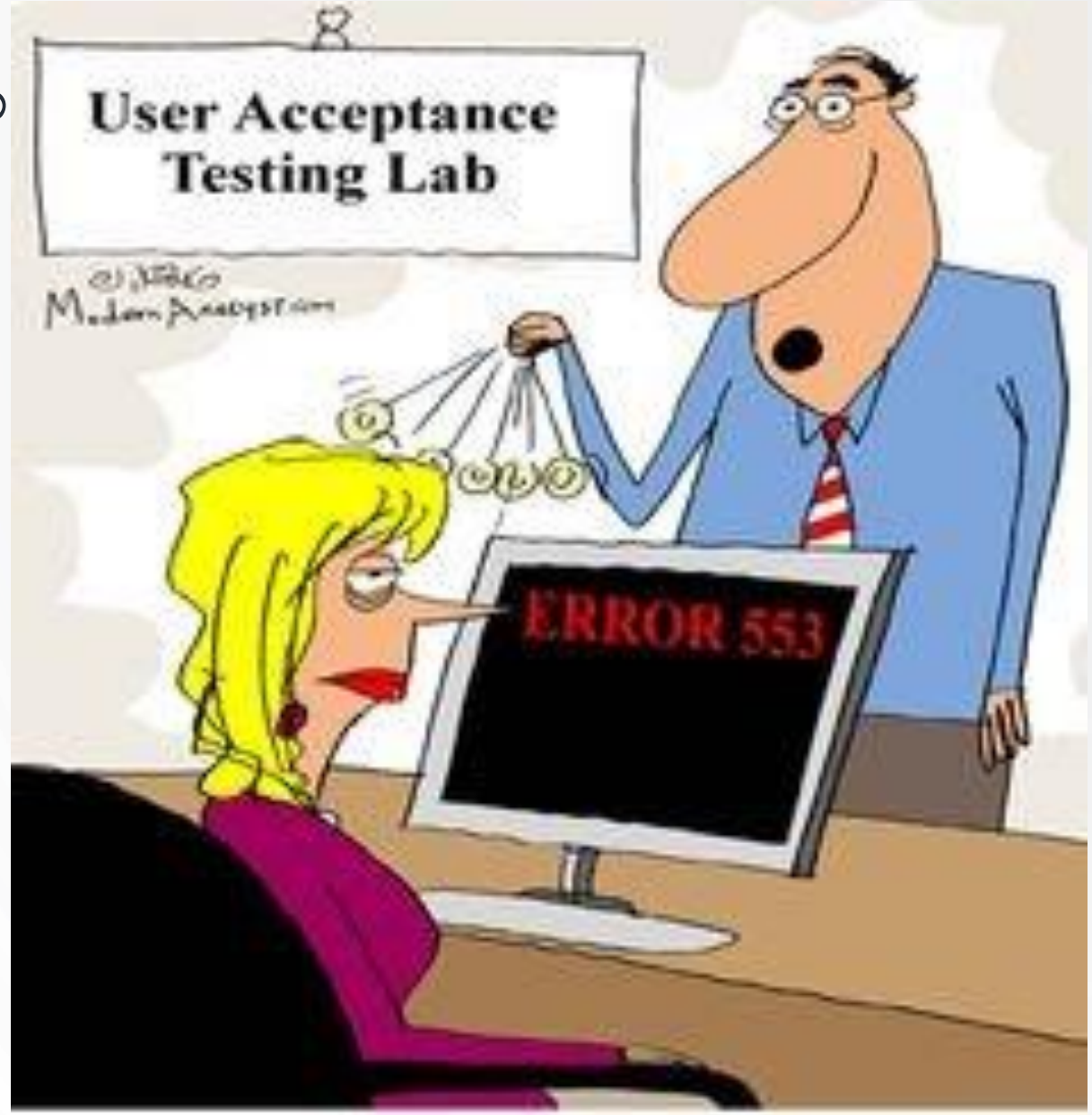
Make sure to fix any breakage

Try and add new tests as needed

Image Courtesy of (https://dzone.com/articles/regression-testing-tools-and-techniques)



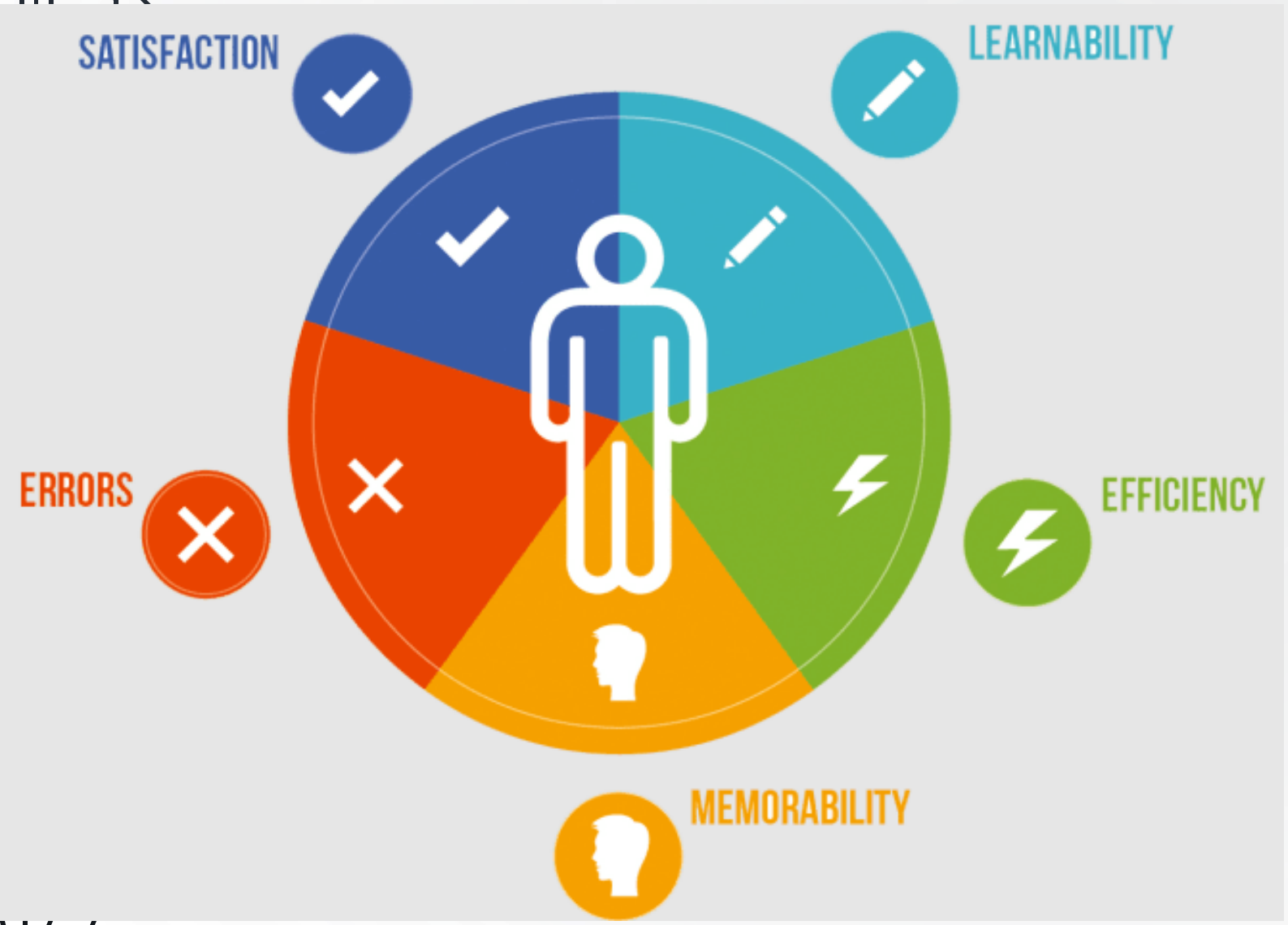
Image" https://www.testingxperts.com/wp-content/uploads/2020/02/types-of-regession-testing-1.jpg



"You love the system! ... You love the system!"

ASD06
Software Testing

A. Defensive programming
B. Automated testing
C. Unit Tests
D. Integration
E. Regression
F. User Acceptance
G. Security



Phase of testing to ensure that the software is ready for production by:

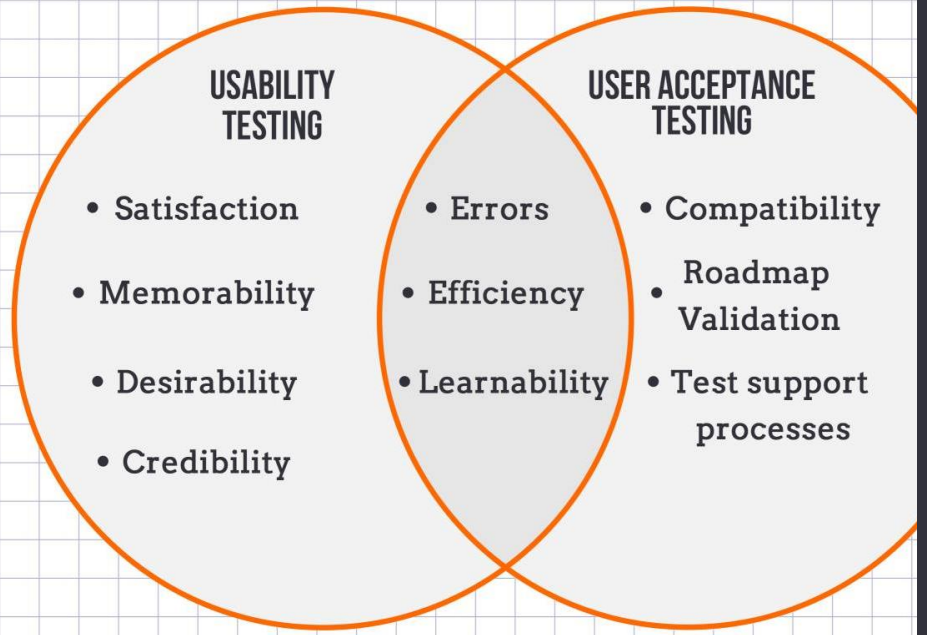
- Making sure it is what the user wanted
- Is usable and user friendly
- Functional: matches user stories (Agile/XP)
- Operational: System can be used and maintained
- Alpha/Beta testing
  are late prototypes usable

https://www.guru99.com/user-acceptance-testing.html

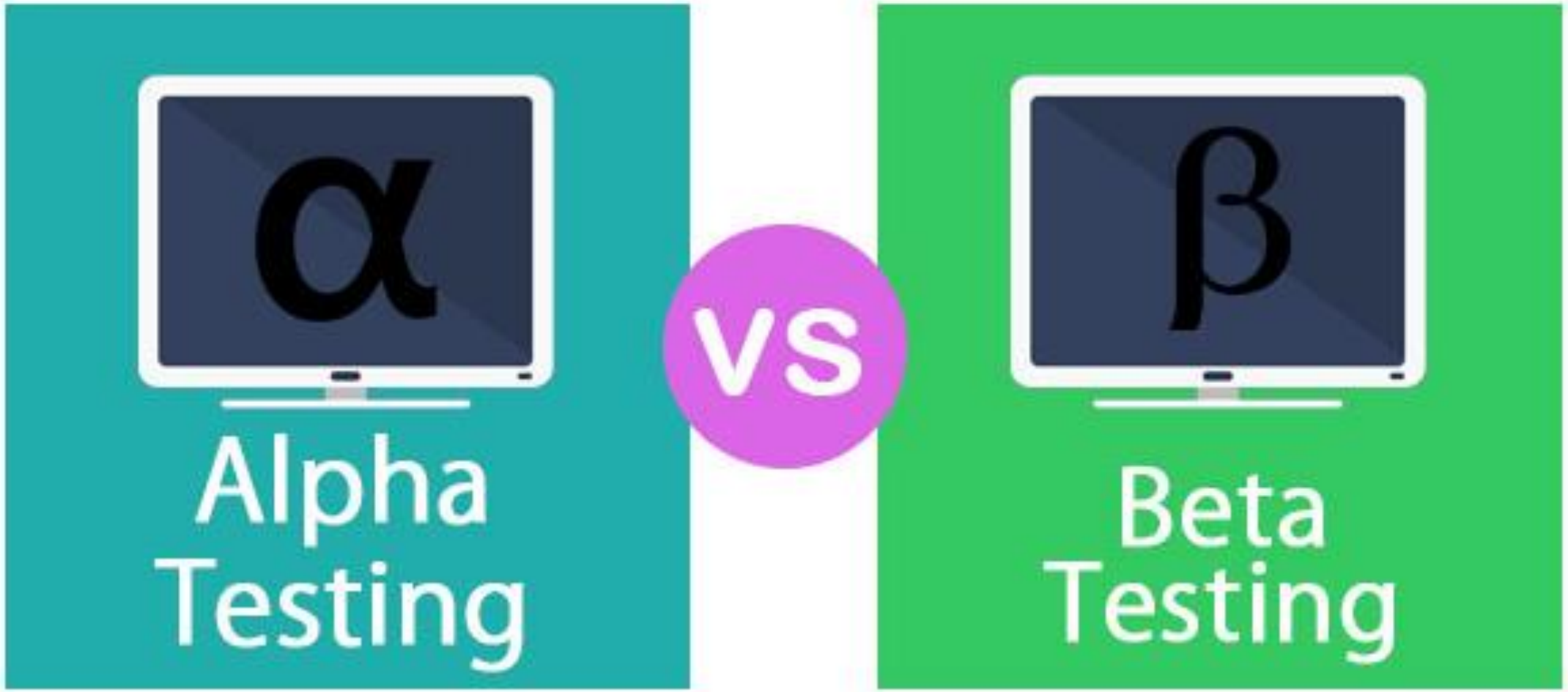
testbytes
Making Quality a Habit

DIFFERENCE BETWEEN

USABILITY & UAT TEST



	USABILITY	USER ACCEPTANCE
1.	Performed to measure the ease of access	Used to validate requirement document from a real user perspective
2.	Done towards the middle and end of SDLC	Performed at the end of the SDLC
3.	Focused on behavior, experience and goals	Focus is on functionality
4.	Can be expensive	Not that expensive



Conducted in software house or with “super users”

Conducted with end-users of the system.

Image: https://www.educba.com/alpha-testing-vs-beta-testing/)





ASD06 Software Testing

A. Defensive programming
B. Automated testing
C. Unit Tests
D. Integration
E. Regression
F. User Acceptance
G. Security

Security Testing



- Vulnerability Scanning
- Security Scanning
- Penetration Testing
- Risk Assessment
- Security Auditing
- Ethical Hacking
- Posture Assessment

www.educba.com

Rewind... 

Security needs to be built-in from the start, e.g.:

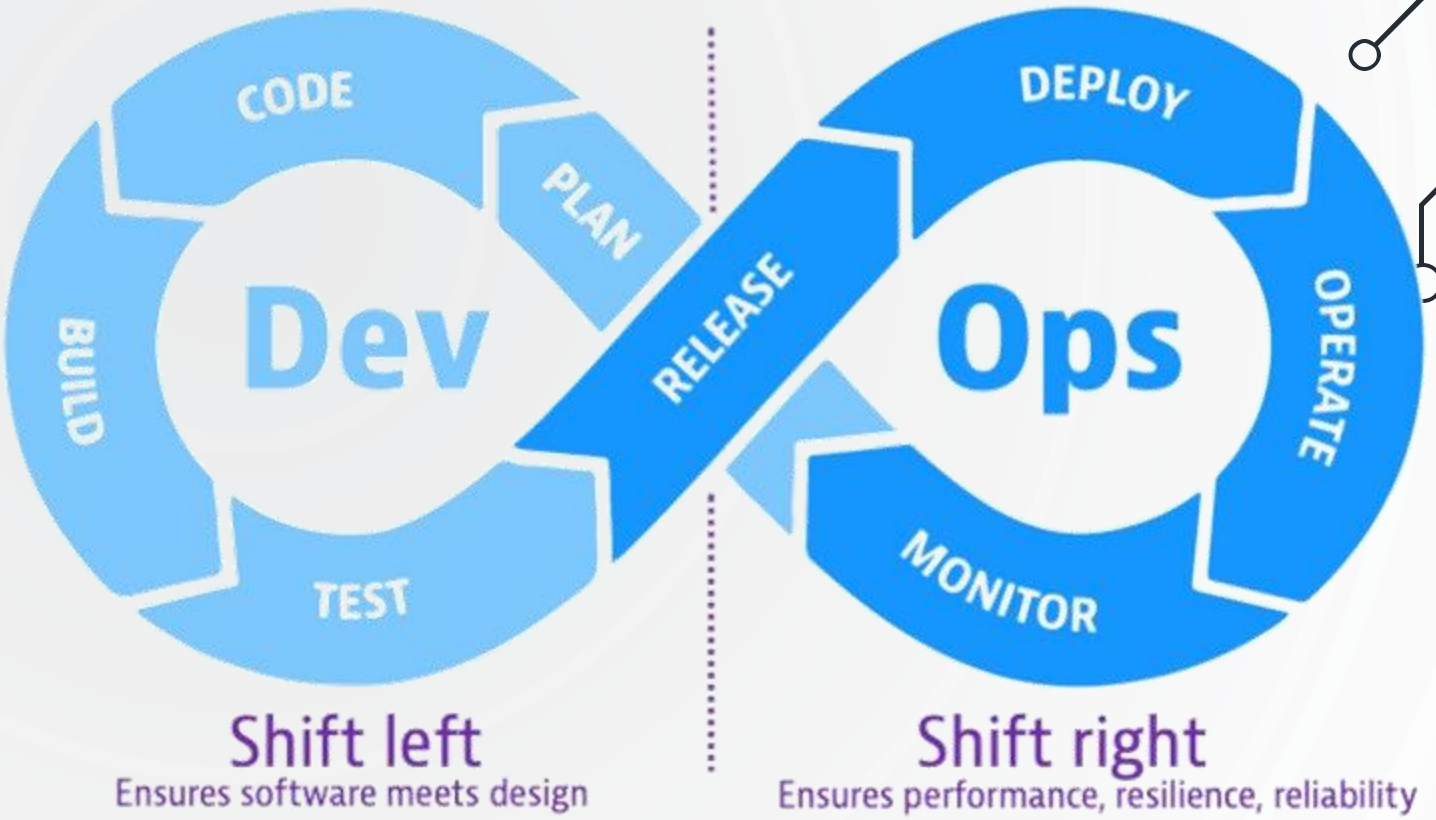
- Parameterised queries, not cut-and-paste SQL ??
- Never display unescaped HTML ??
- Salted password hashes, different salt per uid ??
- Strong cryptographic functions (not RSA, but Kyber) ??
- Do not keep passwords in memory ??

OWASP top 10 vulnerabilities 2021

Problem	Fix
A01- Broken access control (↑5)	Deny by default
A02- Cryptographic fail (=2)	Encrypt data in transit/at rest
A03- Injection (↓1)	Safe API + IAEF: whitelist
A04- Insecure design (↑↑new)	“Shift left”
A05- Security misconfiguration (↑6)	Secure installation process
A06- Vulnerable and outdated components (↑9)	Patch management + signed comp.
A07- Identification and authentication fail (↓2)	Multifactor auth. + no backdoors
A08- Software and data integrity fail (↑↑new)	Signed software
A09- Security logging and monitoring fail (↑10)	Log everything + keep logs
A10- Server-side request forgery (↑↑new)	Sanitise + whitelist

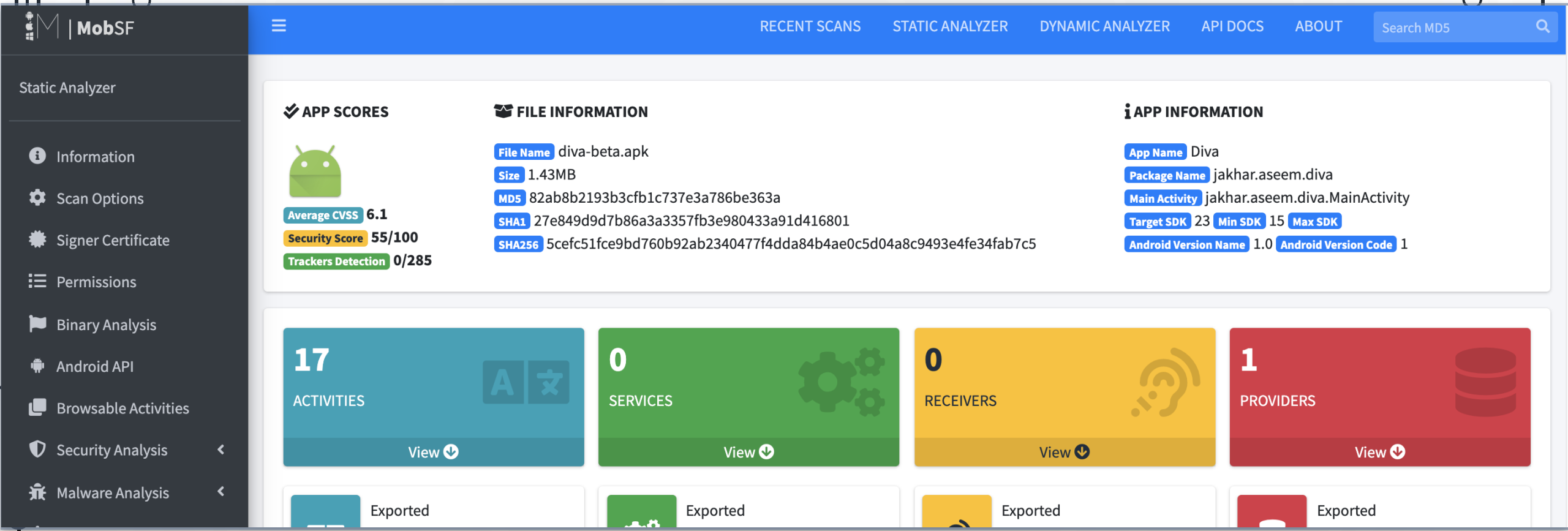
Shift left

- Move testing, security, quality to early in process
- In design: before code
- In code: before test
- Unit tests also test for security & quality issues
- Perform regular sanitizer runs *(cheap and easy)*
  AddressSan, UBSan, MemSan, …



Esp. security is nearly impossible to fix in late stages

Pen testing

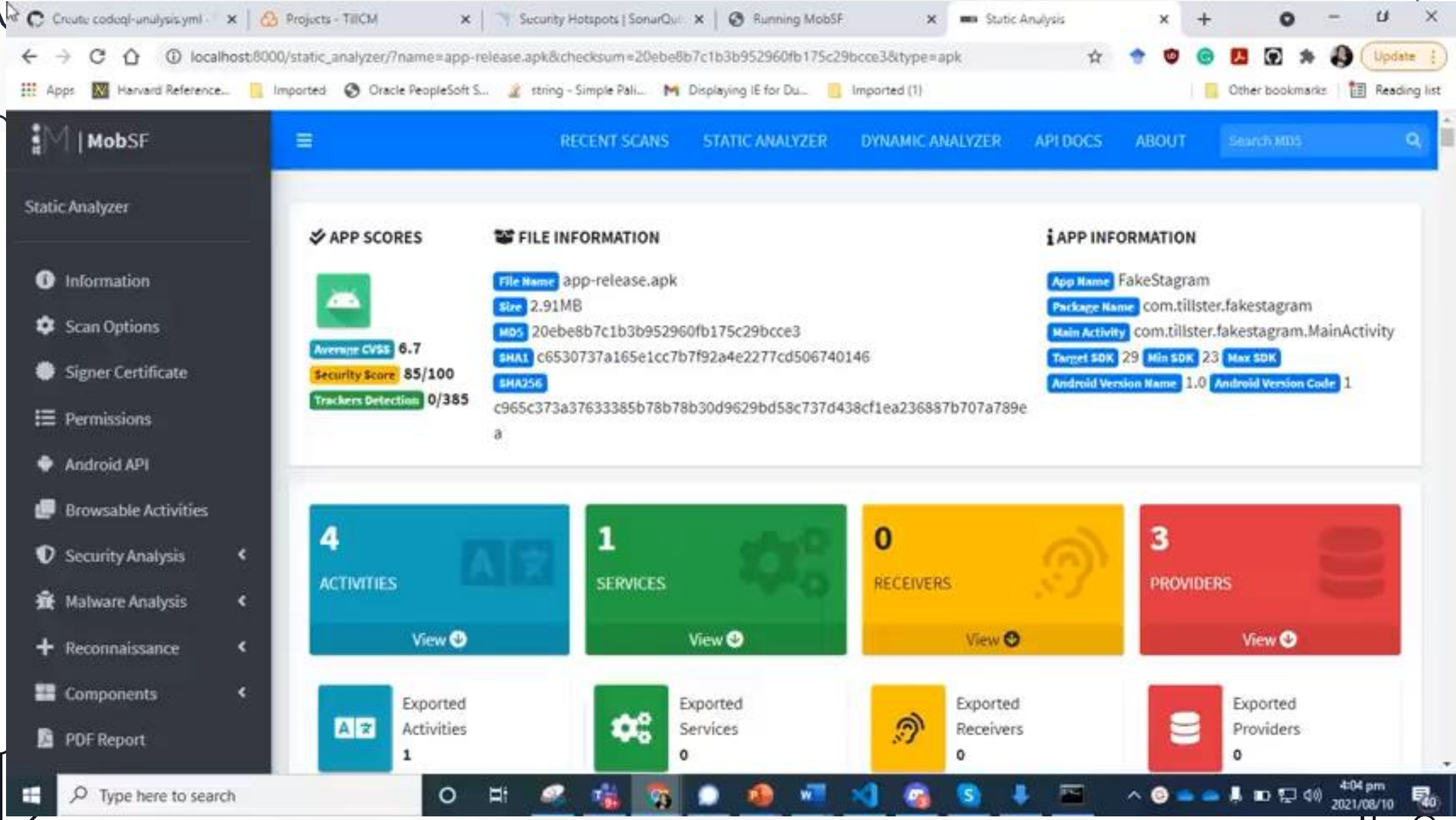


Penetration Testing ("pen testing")

simulates attempts to breach security to discover vulnerabilities.

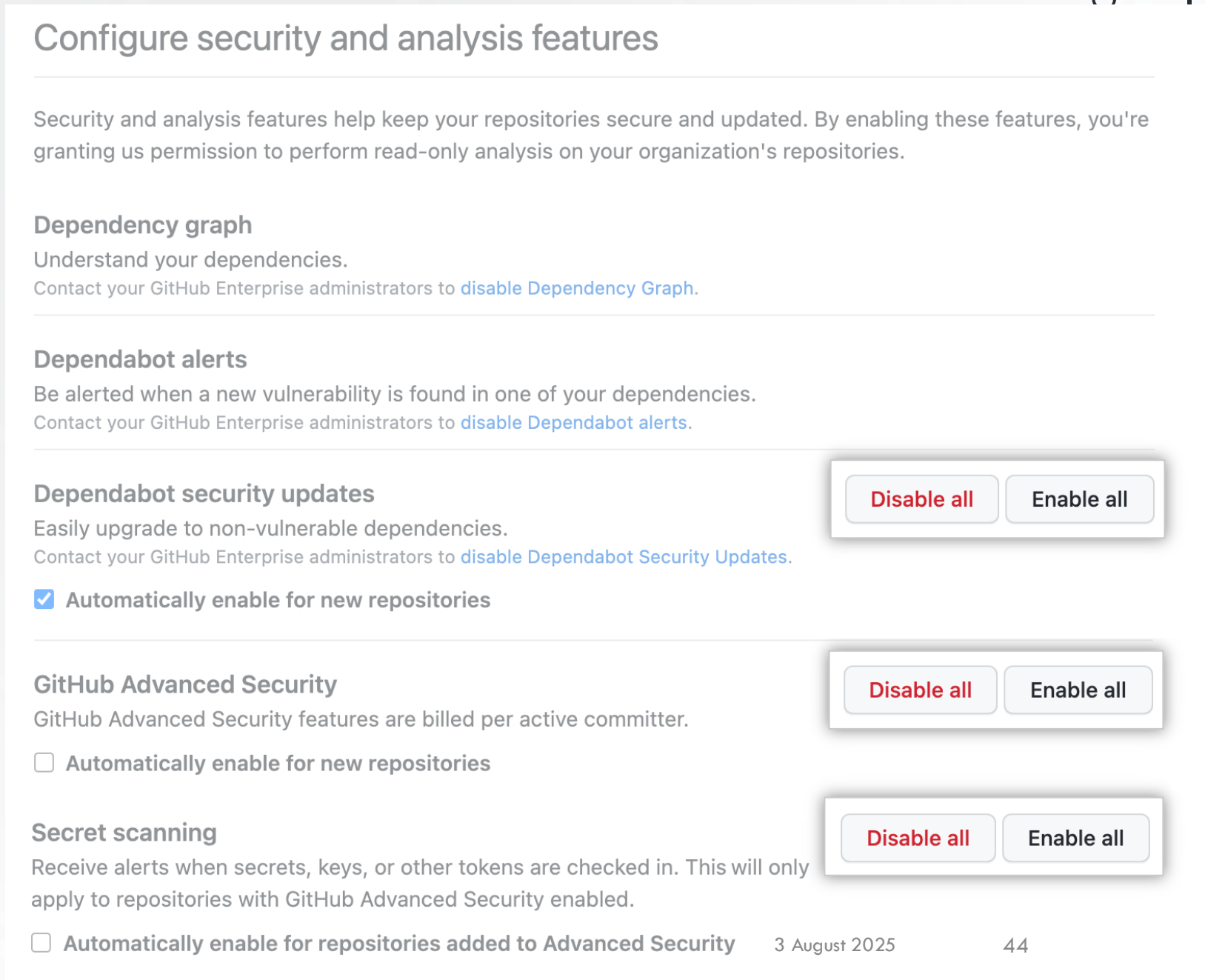
- MobSF is an open-source tool to support pen testing

Example Security Audit



Continuous Integration in GitHub

GitHub allows easy integration with CodeQL to analyse your code for security flaws.



Traps and Pitfalls



It is not enough to be busy, the question is what are we busy about?

– Henry David Thoreau



Absence of proof is not proof of absence.

– William Cowper